Government Data APIs

Week 11 · APIs module · Textbook Chapter 11

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 26, 28 & 30, 2026

This week at a glance

Four public APIs, each showing one idea that holds for every API. This week, every class is live coding, from one lab notebook in three parts.

Monday | Lab, part 1: the Census

- Keys, and *what can I ask for?* — the variable and geography catalogs

Wednesday | Lab, part 2: FRED, then the EIA

- *Let the server compute*, then *let the server filter*

Friday | Lab, part 3: Congress.gov

- *Follow the links*; join two APIs; then textbook revisions

The lab

`labs/week-11/lab-11.ipynb` in the course repository. Each part is added before its day.

Four keys, as environment variables

`CENSUS_KEY` today; `FRED_KEY` and `EIA_KEY` Wednesday; `DATAGOV_KEY` Friday. Links are in the Canvas announcement and the lab's first cell.

Due Sunday, Nov 1

Chapter 11's **Recommended Exercises**, from its companion notebook.

1. Monday · Live-Coding Lab

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-11`, click `lab-11.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment, *after* setting your keys

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–6, about 50 minutes
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-11-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A missing key stops Step 0 *on purpose*: set it and run again. A cell that fails with your key set is *broken*: tell me, or open an issue.

No key yet?

Follow along: the key shows every output. A Census key works only after you click the link in its email.

Public data by mandate

Government APIs exist because of democratic commitments to **transparency** — not business strategy:

- The **Census Bureau** is constitutionally mandated to count the population.
- The **Federal Reserve** publishes the economic data it collects; FRED is the St. Louis Fed's service for it.
- The **Energy Information Administration** was created in 1977, after the energy crises, to publish independent energy statistics.
- The **Library of Congress** publishes Congress's legislative record at Congress.gov.

They embody the **openness** value from Ch. 3, *The Post-API Age*: public data, freely accessible by mandate rather than platform goodwill.

Stable, but less polished

Agencies don't pivot business models or get acquired — so these APIs are unusually **stable**. The tradeoff: dense docs, cryptic variable names, unhelpful errors. Navigating that *is* the skill.

Four APIs, four ideas

Week 10's Wikipedia taught **pagination**. Each API this week teaches one more idea you'll meet in every API:

Day	API	The idea
Monday	Census Bureau	**Discovery**: ask the API what you can ask for
Wednesday	FRED	**Let the server compute**: transforms in the request
Wednesday	EIA	**Let the server filter**: facets narrow the data before it's sent
Friday	Congress.gov	**Follow the links**: records point to related records

And one idea across all four: a **key** says who you are, and it stays out of your notebook.

Keys stay out of the notebook

A **key** identifies you, counts your requests, and enforces rate limits. A key typed into a notebook travels with it: into a commit, a submission, a classmate's laptop.

Set it in your shell *before* starting Jupyter, then read it:

```
# macOS/Linux: export CENSUS_KEY="your-key"
# Windows: setx CENSUS_KEY "your-key"
import os
census_key = os.environ["CENSUS_KEY"] # a missing key stops here
```

The User-Agent still says who sent each request: `WebDataScience/1.0 (your-email@colorado.edu)`.

The chapter's second way

A JSON file, `api_keys.json`, added to `.gitignore` *before your first commit*. A key in git history stays there even after you delete the file.

Names

This course uses `CENSUS_KEY`, `FRED_KEY`, `EIA_KEY`, and `DATAGOV_KEY`. Chapter 11 is being revised to match this week.

Discovery: two catalogs before any data

What | the variable catalog

- Variables come in **groups**: `B25034` is "Year Structure Built"
- `B25034_002E`: group, line, and `E` for estimate (`M` is its margin of error)
- `.../acs5/groups/B25034.json` is itself an API

Where | the geography catalog

- `.../acs5/geography.json` lists every level the dataset answers for
- Each says what it **requires**: a `place` needs a `state`; a `tract` needs a `state` and a `county`
- Places are **FIPS codes**, as strings: Colorado `08`, Boulder `07850`

Codes change between survey years: learn to look them up, not to memorize them. Then **sanity-check** one value: Boulder's median household income is $80,243 (±$3,561), not 7 or 70 million.

Census — from the catalogs to a real place

Name the variables with `get`, the place with `for`, and what it requires with `in`:

```
census_url = "https://api.census.gov/data/2022/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E",
          "for": "place:07850", # Boulder: a FIPS code, kept as a string
          "in": "state:08", # a place requires its state
          "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0]) # row 0 is the header
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric) # values arrive as strings
```

Boulder has 46,551 housing units; 68 were built in 2020 or later, 3,784 in the 2010s.

When there's no estimate: `-666666666`

72 of Colorado's 482 places have no median-income estimate: too few households answered. The API doesn't leave the value out — it sends a number that can't be an income.

Place	People	Median income
Boulder	106,598	80,243
Campo	103	`-666666666`
Bonanza	11	`-666666666`
Carbonate	0	`-666666666`

Average it as is

The mean of that column is −499,979,939. Mask the codes first: `towns.mask(towns < 0)`.

What the codes mean

The Census Bureau's notes on estimate and annotation values list each one; a margin of error of `-222222222` goes with it.

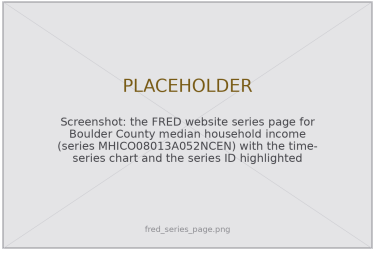
2. Wednesday · Live-Coding Lab

FRED — let the server do the math

Ask FRED for a **transformation** and it returns the result — here, year-over-year percent change:

```
fred_url = ("https://api.stlouisfed.org"
            "/fred/series/observations")

params = {
    "series_id": "CUURS49BSA0", # Denver CPI
    "api_key": fred_key,
    "file_type": "json",
    "units": "pc1", # % change vs. 1yr ago
    "observation_start": "2019-01-01",
}
obs = requests.get(fred_url,
        params=params).json()["observations"]
df = pd.DataFrame(obs)
```



Find series IDs on the FRED website, then copy them *exactly*.

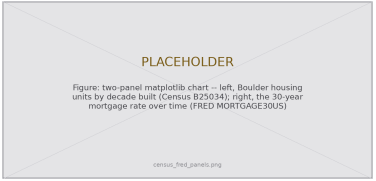
Combine two agencies — and mind the sentinels

The payoff is integration. But guard Census **sentinel values** — suppressed cells return as `-666666666`, not null:

```python
def census_int(v): # NEVER trust a bare int()
    if v is None or str(v).startswith("-6666"):
        return None
    return int(v)

housing = requests.get(census_url,
            params=housing_params).json()
counts = [census_int(housing[1][i])
          for i in range(2, 8)] # by decade

# FRED: 30-yr mortgage rate = the economic "why"
rate = requests.get(fred_url,
          params=mortgage_params).json()
df_rate = pd.DataFrame(rate["observations"])
```



Census says *what* the housing stock is; FRED says *why*.

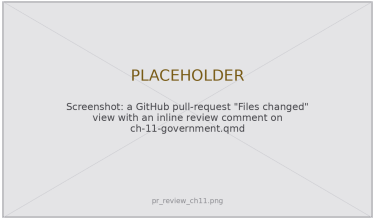
3. Friday · Live-Coding Lab

Improving Chapter 11 together

Today we make the book better. Each of you opens a **pull request** against `ch-11-government.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



High-value targets — pick one and scope it to a single PR:

- **Test the code against live APIs.** Series IDs, endpoints, and election years drift; keys expire. Run the notebook, report what broke, and fix it.
- **Close a gap in "Common Issues to Debug."** Add a worked `-666666666` sentinel-value example, or a rate-limit / retry snippet (FRED ≈120 req/min; api.data.gov 1,000 req/hr).
- **Add a figure.** An annotated variable-code diagram, or the key → request → JSON → DataFrame pipeline.
- **Strengthen a thin exercise.** The six exercises are open-ended — add expected output, a rubric, or a starter cell so they self-check.
- **Extend "Further Reading."** A data-journalism example tied to the **Public Interest** callout — a documented withdrawn dataset or deprecated API.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Social & media platform APIs —** OAuth, Mastodon, and Bluesky in the post-API age.

Key takeaways

1. Government APIs serve **transparency mandates** — valuable, reliable, and unusually stable data.
2. The core skill is **navigating catalogs**, not memorizing codes that change every survey year.
3. **Guard your keys**: environment variables or a git-ignored config — never hardcoded.
4. **Combine sources** for stories no single dataset tells: Census says *what*, FRED says *why*.
5. Mind the **institutional context** — using open data well is a form of civic engagement.